

# Semi-Supervised 3D Cardiac Lead Localization Pipeline

## Research synthesis and recommended modeling strategy

Direct literature on **contact-by-contact cardiac pacing lead localization in 3D CT** is still thin. The closest CT-specific deep learning work I found is mostly about **lead or artifact detection** rather than explicit multiclass segmentation of each electrode contact. Lossau et al. introduced the DyPAR line for **dynamic pacemaker artifact reduction** in cardiac CT using CNNs, and McKeown et al. later developed a CT pipeline that automatically detected ICD lead wires and primary metal artifacts with a 2D U-Net, reporting a Dice score of  $0.958 \pm 0.008$  for artifact detection; their inpainting stage substantially improved downstream chamber segmentation, raising average surface Dice from  $0.684 \pm 0.247$  to  $0.964 \pm 0.067$  and reducing Hausdorff distance from  $3.4 \pm 3.9$  mm to  $0.7 \pm 0.7$  mm. In parallel, large cardiac CT segmentation studies have shown that **coarse localization followed by high-resolution segmentation** is robust at scale, with CNN-based whole-heart pipelines producing clinically useful segmentations across large cohorts. That combination of evidence strongly suggests that, for your dataset, the safest production baseline is a **deep-only, patch-based 3D segmentation pipeline with strong ROI-aware sampling and conservative pseudo-labeling**, not a handcrafted filter stack. <sup>1</sup>

For architecture choice, **nnU-Net-style self-configuring U-Net pipelines remain one of the strongest baselines in medical segmentation**, while transformer hybrids such as UNETR and Swin UNETR have demonstrated state-of-the-art benchmark performance, especially when backed by substantial 3D pretraining. Swin UNETR's strong public results were explicitly tied to large-scale self-supervised pretraining on **5,050 CT volumes**, which matters because your labeled set is only 86 volumes. For that regime, a **residual 3D U-Net or DynUNet-style baseline** is the lower-risk starting point. If your masks later prove to be effectively **point landmarks with only a few positive voxels per class**, then a hybrid upgrade is well motivated: keep segmentation for dense supervision, but add a **heatmap-based landmark head**, because modern 3D landmark detection literature continues to favor **heatmap regression with spatial priors**; the recent nnLandmark framework extends that idea into a self-configuring 3D landmark detector and reports state-of-the-art accuracy across multiple public datasets. <sup>2</sup>

For semi-supervised learning, the field has moved away from naive self-training toward **uncertainty-aware teacher-student, mutual-consistency, dual-view**, and **cross-pseudo-supervision** methods that explicitly suppress noisy pseudo-labels. In volumetric medical segmentation, uncertainty-guided dual-view learning and mutual consistency methods have been especially influential, and recent cardiac-specific work in 2025 also reported gains from **multi-view semi-supervised mean-teacher designs**. Your requested workflow is best implemented as a strict, reproducible self-training loop with **high confidence thresholds, test-time augmentation averaging, entropy filtering, and delayed fine-tuning on accepted pseudo-labels**. That lands very close to the practical center of current semi-supervised best practice, without the engineering overhead of maintaining multiple coupled student networks. <sup>3</sup>

The code below is written as a **single cumulative pipeline**. Each block is meant to live in one file, such as `pipeline.py`, in the order shown.

## Data loading and preprocessing

### Step One: Data Loading and Preprocessing

Your raw label IDs and your training-class order are not the same thing. The code below remaps the raw NIfTI labels `{4001..4009}` into a **contiguous 10-class training index space** where classes **1–6 are electrodes** and classes **7–9 are anatomy**, matching your problem statement exactly. The preprocessing stage uses **nibabel** for NIfTI, supports `.nii.gz`, `.nii`, `.npz`, and gzip-compressed NumPy arrays, normalizes CT intensities, resamples to a common voxel spacing, and caches preprocessed cases as compressed `.npz` files so that every subsequent run is fast and deterministic. The fixed shape is enforced at the **training patch level**, which is safer for tiny electrode targets than globally distorting entire thoracic volumes. Large cardiac CT pipelines in the literature likewise tend to use localization and high-resolution segmentation rather than a single globally resized pass. <sup>4</sup>

```
# =====
# STEP 1: DATA LOADING + PREPROCESSING
# =====
from __future__ import annotations

import gzip
import json
import math
import os
import random
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence, Tuple

import nibabel as nib
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, Dataset
from tqdm.auto import tqdm

import matplotlib.pyplot as plt

from monai.data import list_data_collate
from monai.inferers import sliding_window_inference
from monai.losses import DiceCELoss
```

```

from monai.metrics import HausdorffDistanceMetric
from monai.networks.nets import UNet
from monai.transforms import (
    Compose,
    EnsureChannelFirstd,
    EnsureTyped,
    Rand3DElasticd,
    RandAffined,
    RandCropByLabelClassesd,
    RandFlipd,
    RandRotate90d,
    RandScaleIntensityd,
    RandShiftIntensityd,
)
from monai.utils import set_determinism

# -----
# User-validated raw label map
# -----
RAW_LABEL_MAP: Dict[int, str] = {
    4001: "ANT",          # Anterior wall reference marker
    4002: "Apex",
    4003: "Base",
    4004: "LV_distal",    # LL1
    4005: "LV_2",         # LL2
    4006: "LV_3",         # LL3
    4007: "LV_proximal",  # LL4
    4008: "RV_distal",    # RL1
    4009: "RV_proximal",  # RL2
}

CSV_TO_LABEL: Dict[str, int] = {
    "ANT": 4001,
    "APEX": 4002,
    "BASE": 4003,
    "LL1": 4004,
    "LL2": 4005,
    "LL3": 4006,
    "LL4": 4007,
    "RL1": 4008,
    "RL2": 4009,
}

# -----
# Contiguous training-space class order
# 0 = background
# 1..6 = electrodes

```

```

# 7..9 = anatomy
# -----
CLASS_NAMES: List[str] = [
    "Background",
    "LL1",
    "LL2",
    "LL3",
    "LL4",
    "RL1",
    "RL2",
    "ANT",
    "Apex",
    "Base",
]

RAW_TO_CONTIG: Dict[int, int] = {
    0: 0,
    4004: 1, # LL1
    4005: 2, # LL2
    4006: 3, # LL3
    4007: 4, # LL4
    4008: 5, # RL1
    4009: 6, # RL2
    4001: 7, # ANT
    4002: 8, # Apex
    4003: 9, # Base
}

CONTIG_TO_RAW = {v: k for k, v in RAW_TO_CONTIG.items() if k != 0}

ELECTRODE_CLASSES = [1, 2, 3, 4, 5, 6]
ANATOMY_CLASSES = [7, 8, 9]

@dataclass
class Config:
    # Raw data layout
    raw_labeled_images_dir: str = "data/labeled/images"
    raw_labeled_labels_dir: str = "data/labeled/labels"
    raw_unlabeled_images_dir: str = "data/unlabeled/images"

    # Working directories
    work_dir: str = "runs/cardiac_leads_ssl"

    # Reproducibility
    seed: int = 42

    # Data
    num_classes: int = 10

```

```

labels_already_contiguous: bool = False
target_spacing_dhw: Tuple[float, float, float] = (1.0, 1.0, 1.0)
hu_clip_min: float = -1000.0
hu_clip_max: float = 3000.0

# Training shapes
spatial_dims: int = 3 # set to 2 for slice-wise experiments
patch_size_3d: Tuple[int, int, int] = (160, 160, 160)
train_batch_size: int = 1 # recommended = 1 with patch-sampling + AMP
val_batch_size: int = 1
samples_per_volume: int = 2
num_workers: int = 4
pin_memory: bool = True

# Class-balanced patch sampling ratios for RandCropByLabelClassesd
# [bg, LL1, LL2, LL3, LL4, RL1, RL2, ANT, Apex, Base]
class_sampling_ratios: Tuple[float, ...] = (1, 4, 4, 4, 4, 4, 4, 2, 2, 2)

# Model
channels: Tuple[int, ...] = (32, 64, 128, 256, 512)
strides: Tuple[int, ...] = (2, 2, 2, 2)
num_res_units: int = 2
dropout: float = 0.0

# Optimization
learning_rate: float = 2e-4
weight_decay: float = 1e-5
supervised_epochs: int = 200
finetune_epochs: int = 100
amp: bool = True

# Inference
infer_overlap: float = 0.5
sw_batch_size: int = 2

# Pseudo-labeling
pseudo_min_prob_electrode: float = 0.985
pseudo_min_prob_anatomy: float = 0.95
pseudo_max_entropy: float = 0.20 # normalized entropy threshold
pseudo_min_foreground_voxels: int = 20
pseudo_min_case_mean_conf: float = 0.98
pseudo_weight: float = 0.5

# Loss
lambda_dice: float = 1.0
lambda_ce: float = 1.0

# Metrics

```

```

hausdorff_percentile: Optional[float] = 95.0 # set None for full HD

# Visualization
max_overlay_cases: int = 8

@property
def cache_dir(self) -> Path:
    return Path(self.work_dir) / "cache"

@property
def labeled_cache_dir(self) -> Path:
    return self.cache_dir / "labeled"

@property
def unlabeled_cache_dir(self) -> Path:
    return self.cache_dir / "unlabeled"

@property
def pseudo_cache_dir(self) -> Path:
    return self.cache_dir / "pseudo"

@property
def weights_dir(self) -> Path:
    return Path(self.work_dir) / "weights"

@property
def metrics_dir(self) -> Path:
    return Path(self.work_dir) / "metrics"

@property
def plots_dir(self) -> Path:
    return Path(self.work_dir) / "plots"

@property
def overlays_dir(self) -> Path:
    return Path(self.work_dir) / "overlays"

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    set_determinism(seed=seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

```

```

def ensure_dirs(cfg: Config) -> None:
    for d in [
        cfg.cache_dir,
        cfg.labeled_cache_dir,
        cfg.unlabeled_cache_dir,
        cfg.pseudo_cache_dir,
        cfg.weights_dir,
        cfg.metrics_dir,
        cfg.plots_dir,
        cfg.overlays_dir,
    ]:
        d.mkdir(parents=True, exist_ok=True)

def robust_stem(path: Path) -> str:
    name = path.name
    for suffix in [".nii.gz", ".nii", ".npz", ".npy.gz", ".npy"]:
        if name.endswith(suffix):
            return name[: -len(suffix)]
    return path.stem

def supported_files(directory: str) -> List[Path]:
    directory = Path(directory)
    paths = []
    for ext in [".nii.gz", ".nii", ".npz", ".npy.gz", ".npy"]:
        paths.extend(sorted(directory.glob(ext)))
    return sorted(set(paths))

def pair_labeled_cases(images_dir: str, labels_dir: str) -> List[Tuple[Path, Path]]:
    image_paths = {robust_stem(p): p for p in supported_files(images_dir)}
    label_paths = {robust_stem(p): p for p in supported_files(labels_dir)}
    common = sorted(set(image_paths).intersection(label_paths))
    if not common:
        raise RuntimeError("No paired labeled image/label files were found.")
    return [(image_paths[k], label_paths[k]) for k in common]

def list_unlabeled_cases(images_dir: str) -> List[Path]:
    paths = supported_files(images_dir)
    if not paths:
        raise RuntimeError("No unlabeled image files were found.")
    return paths

def _load_np_numpy_gz(path: Path) -> np.ndarray:

```

```

with gzip.open(path, "rb") as f:
    arr = np.load(f, allow_pickle=False)
    return np.asarray(arr)

def _load_npz(path: Path, key: Optional[str] = None) -> np.ndarray:
    with np.load(path, allow_pickle=False) as data:
        if key is not None and key in data.files:
            return np.asarray(data[key])
        for candidate in ["image", "label", "mask", "arr_0"]:
            if candidate in data.files:
                return np.asarray(data[candidate])
        if len(data.files) == 1:
            return np.asarray(data[data.files[0]])
        raise KeyError(f"Could not infer array key from {path}.")

def load_volume_any(path: Path, array_key: Optional[str] = None) ->
Tuple[np.ndarray, Dict]:
    """
    Returns:
        arr_dhw: np.ndarray with shape [D, H, W]
        meta: dict including spacing_dhw and affine when available
    """
    path = Path(path)
    suffixes = "".join(path.suffixes)

    if suffixes.endswith(".nii.gz") or path.suffix == ".nii":
        nii = nib.as_closest_canonical(nib.load(str(path)))
        arr_xyz = np.asarray(nii.get_fdata(dtype=np.float32))
        if arr_xyz.ndim != 3:
            raise ValueError(f"Expected 3D NIfTI, got shape {arr_xyz.shape}
from {path}.")
        # nibabel returns [X, Y, Z]; DL pipeline uses [D, H, W] == [Z, Y, X]
        arr_dhw = np.transpose(arr_xyz, (2, 1, 0))
        zooms_xyz = np.asarray(nii.header.get_zooms()[:3], dtype=np.float32)
        spacing_dhw = zooms_xyz[[2, 1, 0]]
        meta = {
            "spacing_dhw": spacing_dhw,
            "affine": nii.affine.astype(np.float32),
            "source_path": str(path),
        }
        return arr_dhw, meta

    if path.suffix == ".npz":
        arr = _load_npz(path, key=array_key).astype(np.float32)
        if arr.ndim != 3:
            raise ValueError(f"Expected 3D array in {path}, got shape

```



```

{arr.shape}.")
    meta = {
        "spacing_dhw": np.ones(3, dtype=np.float32),
        "affine": np.eye(4, dtype=np.float32),
        "source_path": str(path),
    }
    return arr.astype(np.float32), meta

if suffixes.endswith(".npy.gz"):
    arr = _load_np_numpy_gz(path).astype(np.float32)
elif path.suffix == ".npy":
    arr = np.load(path, allow_pickle=False).astype(np.float32)
else:
    raise ValueError(f"Unsupported input format: {path}")

if arr.ndim != 3:
    raise ValueError(f"Expected 3D array in {path}, got shape {arr.shape}.")
meta = {
    "spacing_dhw": np.ones(3, dtype=np.float32),
    "affine": np.eye(4, dtype=np.float32),
    "source_path": str(path),
}
return arr, meta

def remap_labels_to_contiguous(label_dhw: np.ndarray, cfg: Config) ->
np.ndarray:
    label_dhw = np.asarray(label_dhw)

    if cfg.labels_already_contiguous:
        valid = set(range(cfg.num_classes))
        uniques = set(np.unique(label_dhw).tolist())
        unknown = sorted(uniques - valid)
        if unknown:
            raise ValueError(f"Found unexpected contiguous labels: {unknown}")
        return label_dhw.astype(np.uint8)

    uniques = set(np.unique(label_dhw).tolist())
    unknown = sorted(uniques - set(RAW_TO_CONTIG.keys()))
    if unknown:
        raise ValueError(f"Found raw labels not in RAW_TO_CONTIG: {unknown}")

    out = np.zeros_like(label_dhw, dtype=np.uint8)
    for raw_id, class_id in RAW_TO_CONTIG.items():
        if raw_id == 0:
            continue
        out[label_dhw == raw_id] = class_id
    return out

```

```

def normalize_ct(image_dhw: np.ndarray, hu_min: float, hu_max: float) ->
np.ndarray:
    image = np.clip(image_dhw.astype(np.float32), hu_min, hu_max)
    image = (image - hu_min) / max(hu_max - hu_min, 1e-8)
    return image.astype(np.float32)

def resample_volume_torch(
    volume_dhw: np.ndarray,
    in_spacing_dhw: Sequence[float],
    out_spacing_dhw: Sequence[float],
    is_label: bool = False,
) -> np.ndarray:
    """
    Resamples a [D, H, W] volume using torch interpolation.
    """
    in_spacing = np.asarray(in_spacing_dhw, dtype=np.float32)
    out_spacing = np.asarray(out_spacing_dhw, dtype=np.float32)

    if np.allclose(in_spacing, out_spacing, atol=1e-5):
        return volume_dhw.astype(np.float32 if not is_label else np.uint8)

    scale = in_spacing / out_spacing
    new_shape = np.maximum(1, np.round(np.asarray(volume_dhw.shape) *
scale).astype(int))

    tensor = torch.from_numpy(volume_dhw.astype(np.float32))[None, None, ...]
    # [1,1,D,H,W]
    mode = "nearest" if is_label else "trilinear"

    with torch.no_grad():
        resized = F.interpolate(
            tensor,
            size=tuple(int(x) for x in new_shape),
            mode=mode,
            align_corners=False if mode == "trilinear" else None,
        )

    out = resized[0, 0].cpu().numpy()
    if is_label:
        out = np.rint(out).astype(np.uint8)
    else:
        out = out.astype(np.float32)
    return out

```

```

def preprocess_case_to_npz(
    image_path: Path,
    output_npz: Path,
    cfg: Config,
    label_path: Optional[Path] = None,
) -> None:
    image_dhw, image_meta = load_volume_any(image_path, array_key="image")
    image_rs = resample_volume_torch(
        image_dhw,
        in_spacing_dhw=image_meta["spacing_dhw"],
        out_spacing_dhw=cfg.target_spacing_dhw,
        is_label=False,
    )
    image_rs = normalize_ct(image_rs, cfg.hu_clip_min, cfg.hu_clip_max)

    payload = {
        "image": image_rs.astype(np.float32),
        "case_id": robust_stem(image_path),
        "spacing_dhw": np.asarray(cfg.target_spacing_dhw, dtype=np.float32),
        "source_image": str(image_path),
    }

    if label_path is not None:
        label_dhw, label_meta = load_volume_any(label_path, array_key="label")
        label_dhw = remap_labels_to_contiguous(label_dhw, cfg)
        label_rs = resample_volume_torch(
            label_dhw,
            in_spacing_dhw=label_meta["spacing_dhw"],
            out_spacing_dhw=cfg.target_spacing_dhw,
            is_label=True,
        )
        payload["label"] = label_rs.astype(np.uint8)
        payload["source_label"] = str(label_path)

    np.savez_compressed(output_npz, **payload)

def build_preprocessed_cache(cfg: Config) -> Dict[str, List[Path]]:
    ensure_dirs(cfg)

    labeled_pairs = pair_labeled_cases(cfg.raw_labeled_images_dir,
    cfg.raw_labeled_labels_dir)
    unlabeled_images = list_unlabeled_cases(cfg.raw_unlabeled_images_dir)

    labeled_npz_paths: List[Path] = []
    for image_path, label_path in tqdm(labeled_pairs, desc="Preprocessing
    labeled"):
        out_path = cfg.labeled_cache_dir / f"{robust_stem(image_path)}.npz"

```

```

        if not out_path.exists():
            preprocess_case_to_npz(image_path, out_path, cfg,
label_path=label_path)
            labeled_npz_paths.append(out_path)

unlabeled_npz_paths: List[Path] = []
for image_path in tqdm(unlabeled_images, desc="Preprocessing unlabeled"):
    out_path = cfg.unlabeled_cache_dir / f"{robust_stem(image_path)}.npz"
    if not out_path.exists():
        preprocess_case_to_npz(image_path, out_path, cfg, label_path=None)
    unlabeled_npz_paths.append(out_path)

meta_path = Path(cfg.work_dir) / "config.json"
with open(meta_path, "w", encoding="utf-8") as f:
    json.dump(asdict(cfg), f, indent=2)

return {
    "labeled": sorted(labeled_npz_paths),
    "unlabeled": sorted(unlabeled_npz_paths),
}

```

## Dataset preparation and augmentation

### Step Two: Dataset Preparation

Because your positive classes are tiny and scattered, the main risk is not simply overfitting, but **feeding the network too many empty patches**. The dataset below uses an **80/20 reproducible split**, a custom PyTorch dataset, and MONAI's **class-balanced patch sampling** so that the loader repeatedly sees lead-contact voxels instead of mostly background. The heavier spatial transforms are then applied on-device through CUDA-friendly tensor operations, which keeps the augmentation policy close to current practice for sparse 3D targets. Small-target and scattered-target segmentation remain a recognized challenge in medical imaging, and semi-supervised 3D methods have also emphasized hard or ambiguous regions for exactly this reason. <sup>5</sup>

```

# =====
# STEP 2: DATASET PREPARATION + AUGMENTATION
# =====

class NPZSegmentationDataset(Dataset):
    """
    Loads preprocessed .npz cases.
    Supports:
        - image only
        - image + label
        - metadata fields for later saving/evaluation
    """

```

```

def __init__(
    self,
    npz_paths: Sequence[Path],
    pseudo_flags: Optional[Sequence[bool]] = None,
    transform: Optional[Compose] = None,
    with_label: bool = True,
) -> None:
    self.npz_paths = [Path(p) for p in npz_paths]
    self.pseudo_flags = list(pseudo_flags) if pseudo_flags is not None else
[False] * len(self.npz_paths)
    if len(self.pseudo_flags) != len(self.npz_paths):
        raise ValueError("pseudo_flags must have same length as npz_paths")
    self.transform = transform
    self.with_label = with_label

def __len__(self) -> int:
    return len(self.npz_paths)

def __getitem__(self, index: int):
    path = self.npz_paths[index]
    with np.load(path, allow_pickle=False) as data:
        image = data["image"].astype(np.float32)
        case_id = str(data["case_id"]) if "case_id" in data.files else
path.stem
        sample = {"image": image}
        if self.with_label:
            if "label" not in data.files:
                raise KeyError(f"{path} does not contain label")
            sample["label"] = data["label"].astype(np.int64)

    base_meta = {
        "case_id": case_id,
        "npz_path": str(path),
        "is_pseudo": torch.tensor(float(self.pseudo_flags[index]),
dtype=torch.float32),
    }

    if self.transform is not None:
        sample = self.transform(sample)

    if isinstance(sample, list):
        out = []
        for item in sample:
            item.update(base_meta)
            out.append(item)
        return out

    sample.update(base_meta)

```

```

        return sample

def build_train_transform(cfg: Config) -> Compose:
    return Compose(
        [
            EnsureChannelFirstd(keys=["image", "label"],
channel_dim="no_channel"),
            RandCropByLabelClassesd(
                keys=["image", "label"],
                label_key="label",
                spatial_size=cfg.patch_size_3d,
                ratios=cfg.class_sampling_ratios,
                num_classes=cfg.num_classes,
                num_samples=cfg.samples_per_volume,
            ),
            EnsureTyped(keys=["image", "label"], dtype=(torch.float32,
torch.int64), track_meta=False),
        ]
    )

def build_val_transform() -> Compose:
    return Compose(
        [
            EnsureChannelFirstd(keys=["image", "label"],
channel_dim="no_channel"),
            EnsureTyped(keys=["image", "label"], dtype=(torch.float32,
torch.int64), track_meta=False),
        ]
    )

def build_unlabeled_transform() -> Compose:
    return Compose(
        [
            EnsureChannelFirstd(keys=["image"], channel_dim="no_channel"),
            EnsureTyped(keys=["image"], dtype=torch.float32, track_meta=False),
        ]
    )

def build_gpu_train_augment(cfg: Config) -> Compose:
    """
    These MONAI transforms operate on torch tensors.
    They are applied after batches are moved to CUDA.
    """
    return Compose(

```

```

[
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=0),
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=1),
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=2),
    RandRotate90d(keys=["image", "label"], prob=0.25, max_k=3,
spatial_axes=(1, 2)),
    RandAffined(
        keys=["image", "label"],
        prob=0.25,
        rotate_range=(0.12, 0.12, 0.12),
        scale_range=(0.10, 0.10, 0.10),
        mode=("bilinear", "nearest"),
        padding_mode="border",
    ),
    Rand3DElasticd(
        keys=["image", "label"],
        prob=0.10,
        sigma_range=(4, 7),
        magnitude_range=(20, 50),
        mode=("bilinear", "nearest"),
        padding_mode="border",
    ),
    RandScaleIntensityd(keys=["image"], factors=0.10, prob=0.20),
    RandShiftIntensityd(keys=["image"], offsets=0.10, prob=0.20),
]
)

```

```

def apply_gpu_augment(
    images: torch.Tensor,
    labels: torch.Tensor,
    gpu_aug: Optional[Compose],
) -> Tuple[torch.Tensor, torch.Tensor]:
    if gpu_aug is None:
        return images, labels

    aug_images = []
    aug_labels = []
    for i in range(images.shape[0]):
        sample = {
            "image": images[i],
            "label": labels[i],
        }
        sample = gpu_aug(sample)
        aug_images.append(sample["image"])
        aug_labels.append(sample["label"])

    return torch.stack(aug_images, dim=0), torch.stack(aug_labels, dim=0)

```

```

def create_train_val_split(
    labeled_npz_paths: Sequence[Path],
    seed: int,
    val_fraction: float = 0.20,
) -> Tuple[List[Path], List[Path]]:
    train_files, val_files = train_test_split(
        list(labeled_npz_paths),
        test_size=val_fraction,
        random_state=seed,
        shuffle=True,
    )
    return sorted(train_files), sorted(val_files)

def create_train_loader(
    npz_paths: Sequence[Path],
    pseudo_flags: Sequence[bool],
    cfg: Config,
) -> DataLoader:
    dataset = NPZSegmentationDataset(
        npz_paths=npz_paths,
        pseudo_flags=pseudo_flags,
        transform=build_train_transform(cfg),
        with_label=True,
    )
    return DataLoader(
        dataset,
        batch_size=cfg.train_batch_size,
        shuffle=True,
        num_workers=cfg.num_workers,
        pin_memory=cfg.pin_memory,
        persistent_workers=cfg.num_workers > 0,
        collate_fn=list_data_collate,
    )
# needed because RandCropByLabelClassesd emits lists

def create_val_loader(
    npz_paths: Sequence[Path],
    cfg: Config,
) -> DataLoader:
    dataset = NPZSegmentationDataset(
        npz_paths=npz_paths,
        pseudo_flags=[False] * len(npz_paths),
        transform=build_val_transform(),
        with_label=True,
    )

```



```

    )
    return DataLoader(
        dataset,
        batch_size=cfg.val_batch_size,
        shuffle=False,
        num_workers=max(1, cfg.num_workers // 2),
        pin_memory=cfg.pin_memory,
        persistent_workers=(max(1, cfg.num_workers // 2) > 0),
    )

def create_unlabeled_loader(
    npz_paths: Sequence[Path],
    cfg: Config,
) -> DataLoader:
    dataset = NPZSegmentationDataset(
        npz_paths=npz_paths,
        pseudo_flags=[False] * len(npz_paths),
        transform=build_unlabeled_transform(),
        with_label=False,
    )
    return DataLoader(
        dataset,
        batch_size=1,
        shuffle=False,
        num_workers=max(1, cfg.num_workers // 2),
        pin_memory=cfg.pin_memory,
        persistent_workers=(max(1, cfg.num_workers // 2) > 0),
    )

```

## Model definition

### Step Three: Model Definition

Given your label count, task sparsity, and dataset size, I would start with a **residual MONAI U-Net**. That stays close to the most reliable medical segmentation baselines and avoids asking a small CT dataset to fully feed a transformer from scratch. If you later add large-scale 3D pretraining, **UNETR or Swin UNETR** become reasonable upgrade paths; without that, a strong CNN U-Net is still the better first model. The implementation below exposes a `spatial_dims` switch so the same pipeline can run in **2D or 3D**. <sup>6</sup>

```

# =====
# STEP 3: MODEL DEFINITION
# =====

def build_model(cfg: Config) -> torch.nn.Module:
    """

```

```

Flexible MONAI U-Net that can toggle between 2D and 3D.
"""
if cfg.spatial_dims not in (2, 3):
    raise ValueError("cfg.spatial_dims must be 2 or 3")

model = UNet(
    spatial_dims=cfg.spatial_dims,
    in_channels=1,
    out_channels=cfg.num_classes,
    channels=cfg.channels,
    strides=cfg.strides,
    num_res_units=cfg.num_res_units,
    dropout=cfg.dropout,
    norm="INSTANCE",
)
return model

def prepare_training_batch_for_model(
    images: torch.Tensor,
    labels: torch.Tensor,
    cfg: Config,
    training: bool = True,
) -> Tuple[torch.Tensor, torch.Tensor]:
    """
    3D mode:
        input remains [B, 1, D, H, W]
    2D mode:
        select one informative axial slice from each 3D patch
        output becomes [B, 1, H, W]
    """
    if cfg.spatial_dims == 3:
        return images, labels

    # Slice-wise 2D mode
    selected_images = []
    selected_labels = []

    for img, lbl in zip(images, labels):
        # img/lbl shapes: [1, D, H, W]
        fg_per_slice = (lbl[0] > 0).sum(dim=(1, 2)) # [D]

        if fg_per_slice.max() > 0:
            valid_z = torch.where(fg_per_slice > 0)[0]
            if training:
                z_idx = valid_z[torch.randint(0, len(valid_z), (1,)),
device=valid_z.device)].item()
            else:

```

```

        z_idx = int(valid_z[len(valid_z) // 2].item())
    else:
        z_idx = img.shape[1] // 2

    selected_images.append(img[:, z_idx, :, :]) # [1, H, W]
    selected_labels.append(lbl[:, z_idx, :, :]) # [1, H, W]

    return torch.stack(selected_images, dim=0), torch.stack(selected_labels,
dim=0)

@torch.no_grad()
def infer_logits(
    model: torch.nn.Module,
    images: torch.Tensor,
    cfg: Config,
) -> torch.Tensor:
    """
    3D mode:
        sliding-window inference over the full volume
    2D mode:
        infer slice-by-slice and reconstruct a 3D logits volume
    Returns:
        logits of shape [B, C, D, H, W]
    """
    if cfg.spatial_dims == 3:
        return sliding_window_inference(
            inputs=images,
            roi_size=cfg.patch_size_3d,
            sw_batch_size=cfg.sw_batch_size,
            predictor=model,
            overlap=cfg.infer_overlap,
        )

    # 2D model: reconstruct 3D volume by running every axial slice
    b, c, d, h, w = images.shape
    logits_slices = []
    for z in range(d):
        logits_2d = model(images[:, :, z, :, :]) # [B, C, H, W]
        logits_slices.append(logits_2d.unsqueeze(2)) # [B, C, 1, H, W]
    return torch.cat(logits_slices, dim=2)

```

## Training and semi-supervised learning

### Step Four: Training Setup

For supervised training, the combined loss is

$$\mathcal{L}_{\text{sup}} = \lambda_{\text{Dice}} \mathcal{L}_{\text{Dice}}(\hat{y}, y) + \lambda_{\text{CE}} \mathcal{L}_{\text{CE}}(\hat{y}, y),$$

where  $\hat{y}$  is the network softmax output and  $y$  is the integer segmentation mask. Because your positive classes are small, the CE term is class-weighted from the empirical training distribution, while the Dice term ignores background to keep optimization focused on the actual targets.

### Step Five: Semi-Supervised Learning

For pseudo-labeling, the pipeline first trains on the 86 labeled cases, then runs full-volume inference on the 130 unlabeled scans. It uses **test-time augmentation averaging** and **entropy filtering** to make pseudo-labels less brittle. The hard pseudo-label at voxel  $v$  is

$$\tilde{y}_v = \arg \max_c p(c | x)_v,$$

and it is accepted only if

$$\max_c p(c | x)_v \geq \tau_c \quad \text{and} \quad H(p(\cdot | x)_v) \leq \eta,$$

where  $\tau_c$  is a class-specific confidence threshold and  $H(\cdot)$  is normalized predictive entropy. This is a simpler production implementation of the same core idea behind current uncertainty-aware semi-supervised volumetric segmentation methods. 3

```
# =====
# STEPS 4 + 5: TRAINING + SEMI-SUPERVISED PSEUDO-LABELING
# =====

def compute_class_weights(
    train_npz_paths: Sequence[Path],
    num_classes: int,
) -> torch.Tensor:
    counts = np.zeros(num_classes, dtype=np.float64)
    for path in train_npz_paths:
        with np.load(path, allow_pickle=False) as data:
            y = data["label"].astype(np.int64)
            unique, freq = np.unique(y, return_counts=True)
            counts[unique] += freq

    counts = np.maximum(counts, 1.0)
    weights = counts.sum() / counts
    weights = weights / weights.mean()

    # So background does not dominate the CE term
    weights[0] *= 0.25
    return torch.tensor(weights, dtype=torch.float32)
```

```

def build_loss_fn(class_weights: torch.Tensor, cfg: Config) -> DiceCELoss:
    return DiceCELoss(
        include_background=False, # for Dice only
        to_onehot_y=True,
        softmax=True,
        ce_weight=class_weights,
        lambda_dice=cfg.lambda_dice,
        lambda_ce=cfg.lambda_ce,
        squared_pred=False,
        smooth_nr=1e-5,
        smooth_dr=1e-5,
    )

def labels_to_one_hot(labels: torch.Tensor, num_classes: int) -> torch.Tensor:
    # labels: [B, 1, D, H, W] or [B, 1, H, W]
    squeezed = labels.squeeze(1).long()
    onehot = F.one_hot(squeezed, num_classes=num_classes)
    dims = list(range(onehot.ndim))
    onehot = onehot.permute(0, dims[-1], *dims[1:-1]).float()
    return onehot

@torch.no_grad()
def mean_dice_from_logits(
    logits: torch.Tensor,
    labels: torch.Tensor,
    num_classes: int,
    include_background: bool = False,
) -> float:
    preds = torch.argmax(logits, dim=1, keepdim=True)
    pred_oh = labels_to_one_hot(preds, num_classes)
    true_oh = labels_to_one_hot(labels, num_classes)

    reduce_dims = tuple(range(2, pred_oh.ndim))
    inter = (pred_oh * true_oh).sum(dim=reduce_dims)
    denom = pred_oh.sum(dim=reduce_dims) + true_oh.sum(dim=reduce_dims)

    dice = torch.where(
        denom > 0,
        (2.0 * inter + 1e-6) / (denom + 1e-6),
        torch.full_like(denom, torch.nan),
    )

    if not include_background:
        dice = dice[:, 1:]

    return float(torch.nanmean(dice).item())

```

```

def save_checkpoint(
    path: Path,
    model: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
    epoch: int,
    best_metric: float,
    cfg: Config,
) -> None:
    payload = {
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "epoch": epoch,
        "best_metric": best_metric,
        "config": asdict(cfg),
    }
    torch.save(payload, path)

def train_one_epoch(
    model: torch.nn.Module,
    loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    loss_fn: DiceCELoss,
    scaler: torch.cuda.amp.GradScaler,
    device: torch.device,
    cfg: Config,
    gpu_aug: Optional[Compose] = None,
) -> float:
    model.train()
    epoch_losses: List[float] = []

    for batch in tqdm(loader, desc="Train", leave=False):
        images = batch["image"].to(device, non_blocking=True)
        labels = batch["label"].to(device, non_blocking=True).long()

        if gpu_aug is not None:
            images, labels = apply_gpu_augment(images, labels, gpu_aug)

        model_inputs, model_labels = prepare_training_batch_for_model(
            images=images,
            labels=labels,
            cfg=cfg,
            training=True,
        )

        optimizer.zero_grad(set_to_none=True)

```

```

        with torch.autocast(device_type=device.type, enabled=(cfg.amp and
device.type == "cuda")):
            logits = model(model_inputs)
            loss = loss_fn(logits, model_labels)

            # If this batch came from pseudo-labeled data, downweight it.
            # With train_batch_size=1, all patches in the batch are from the
            same source case.
            batch_is_pseudo = bool(batch["is_pseudo"][0].item()) if "is_pseudo"
in batch else False
            if batch_is_pseudo:
                loss = cfg.pseudo_weight * loss

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

            epoch_losses.append(float(loss.item()))

    return float(np.mean(epoch_losses)) if epoch_losses else np.nan

@torch.no_grad()
def validate_one_epoch(
    model: torch.nn.Module,
    loader: DataLoader,
    loss_fn: DiceCELoss,
    device: torch.device,
    cfg: Config,
) -> Dict[str, float]:
    model.eval()

    losses: List[float] = []
    dices: List[float] = []

    for batch in tqdm(loader, desc="Validate", leave=False):
        images = batch["image"].to(device, non_blocking=True)
        labels = batch["label"].to(device, non_blocking=True).long()

        with torch.autocast(device_type=device.type, enabled=(cfg.amp and
device.type == "cuda")):
            logits = infer_logits(model, images, cfg)
            loss = loss_fn(logits, labels)

        losses.append(float(loss.item()))
        dices.append(mean_dice_from_logits(logits, labels, cfg.num_classes,
include_background=False))

```

```

    return {
        "val_loss": float(np.mean(losses)) if losses else np.nan,
        "val_dice": float(np.mean(dices)) if dices else np.nan,
    }

def fit_model(
    model: torch.nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    loss_fn: DiceCELoss,
    device: torch.device,
    cfg: Config,
    epochs: int,
    checkpoint_path: Path,
    stage_name: str,
    gpu_aug: Optional[Compose] = None,
) -> pd.DataFrame:
    best_dice = -np.inf
    history_rows: List[Dict] = []
    scaler = torch.cuda.amp.GradScaler(enabled=(cfg.amp and device.type ==
"cuda"))

    for epoch in range(1, epochs + 1):
        train_loss = train_one_epoch(
            model=model,
            loader=train_loader,
            optimizer=optimizer,
            loss_fn=loss_fn,
            scaler=scaler,
            device=device,
            cfg=cfg,
            gpu_aug=gpu_aug,
        )

        val_stats = validate_one_epoch(
            model=model,
            loader=val_loader,
            loss_fn=loss_fn,
            device=device,
            cfg=cfg,
        )

        row = {
            "stage": stage_name,
            "epoch": epoch,

```



```

        "train_loss": train_loss,
        "val_loss": val_stats["val_loss"],
        "val_dice": val_stats["val_dice"],
    }
    history_rows.append(row)

    if val_stats["val_dice"] > best_dice:
        best_dice = val_stats["val_dice"]
        save_checkpoint(
            path=checkpoint_path,
            model=model,
            optimizer=optimizer,
            epoch=epoch,
            best_metric=best_dice,
            cfg=cfg,
        )

    print(
        f"[{stage_name}] epoch={epoch:03d} "
        f"train_loss={train_loss:.5f} "
        f"val_loss={val_stats['val_loss']:.5f} "
        f"val_dice={val_stats['val_dice']:.5f}"
    )

    history_df = pd.DataFrame(history_rows)
    history_df.to_csv(Path(cfg.work_dir) / f"history_{stage_name}.csv",
                      index=False)
    return history_df

@torch.no_grad()
def tta_mean_probabilities(
    model: torch.nn.Module,
    image: torch.Tensor,
    cfg: Config,
) -> Tuple[torch.Tensor, torch.Tensor]:
    """
    Returns:
        mean_probs: [B, C, D, H, W]
        normalized_entropy: [B, D, H, W]
    """
    flip_sets = [
        tuple(),
        (2,), # D
        (3,), # H
        (4,), # W
    ]

```

```

probs_all = []
for dims in flip_sets:
    aug_img = torch.flip(image, dims=dims) if dims else image
    logits = infer_logits(model, aug_img, cfg)
    probs = torch.softmax(logits, dim=1)
    if dims:
        probs = torch.flip(probs, dims=dims)
    probs_all.append(probs)

prob_stack = torch.stack(probs_all, dim=0)
mean_probs = prob_stack.mean(dim=0)

entropy = -(mean_probs.clamp_min(1e-8) *
mean_probs.clamp_min(1e-8).log()).sum(dim=1)
entropy = entropy / math.log(cfg.num_classes) # normalize to [0, 1]
approximately
return mean_probs, entropy

def class_threshold_tensor(device: torch.device, cfg: Config) -> torch.Tensor:
    thresholds = torch.ones(cfg.num_classes, device=device) *
cfg.pseudo_min_prob_anatomy
    thresholds[0] = 1.0
    for c in ELECTRODE_CLASSES:
        thresholds[c] = cfg.pseudo_min_prob_electrode
    for c in ANATOMY_CLASSES:
        thresholds[c] = cfg.pseudo_min_prob_anatomy
    return thresholds

@torch.no_grad()
def generate_pseudo_labels(
    model: torch.nn.Module,
    unlabeled_loader: DataLoader,
    device: torch.device,
    cfg: Config,
) -> List[Path]:
    model.eval()
    accepted_pseudo_files: List[Path] = []
    thresholds = class_threshold_tensor(device, cfg)
    stats_rows: List[Dict] = []

    cfg.pseudo_cache_dir.mkdir(parents=True, exist_ok=True)

    for batch in tqdm(unlabeled_loader, desc="Pseudo-labeling"):
        image_cpu = batch["image"] # keep CPU copy for saving
        image = batch["image"].to(device, non_blocking=True)
        case_id = batch["case_id"][0]

```

```

source_npz_path = Path(batch["npz_path"][0])

mean_probs, entropy = tta_mean_probabilities(model, image, cfg)
conf, pred = mean_probs.max(dim=1) # [B, D, H, W]

per_voxel_threshold = thresholds[pred]
confident = (conf >= per_voxel_threshold) & (entropy <=
cfg.pseudo_max_entropy)

pseudo = torch.where(confident, pred, torch.zeros_like(pred))
fg_mask = pseudo > 0

fg_voxels = int(fg_mask.sum().item())
case_mean_conf = float(conf[fg_mask].mean().item()) if fg_voxels > 0
else 0.0

accept = (
    fg_voxels >= cfg.pseudo_min_foreground_voxels
    and case_mean_conf >= cfg.pseudo_min_case_mean_conf
)

stats_rows.append(
    {
        "case_id": case_id,
        "fg_voxels": fg_voxels,
        "pseudo_mean_conf": case_mean_conf,
        "accepted": int(accept),
        "source_npz": str(source_npz_path),
    }
)

if not accept:
    continue

out_path = cfg.pseudo_cache_dir / f"{case_id}_pseudo.npz"
np.savez_compressed(
    out_path,
    image=image_cpu[0, 0].cpu().numpy().astype(np.float32),
    label=pseudo[0].cpu().numpy().astype(np.uint8),
    case_id=case_id,
    source_image_npz=str(source_npz_path),
    pseudo_mean_conf=np.float32(case_mean_conf),
)
accepted_pseudo_files.append(out_path)

pd.DataFrame(stats_rows).to_csv(Path(cfg.work_dir) /
"pseudo_label_stats.csv", index=False)
return sorted(accepted_pseudo_files)

```

## Evaluation metrics and quantitative plots

### Step Six: Evaluation Metrics

For this task, relying on a single mean Dice is too weak. Your targets are **small, sparse, and clinically localized**, so the evaluation must expose both overlap quality and boundary quality. The code below computes per-class and mean Dice, IoU, per-electrode precision/recall/F1, a full pixel-wise confusion matrix, and Hausdorff distance. That choice matches the way boundary-sensitive cardiac CT studies and lead-artifact papers are typically evaluated, where surface or boundary errors matter disproportionately near devices and thin structures. <sup>7</sup>

### Step Seven: Quantitative Plots

The plotting block saves all required figures: training and validation loss, validation Dice by epoch, a validation-sample Dice boxplot, per-class Dice bars, a voxel-level precision-recall curve for electrode detection, and a confusion-matrix heatmap.

```
# =====
# STEPS 6 + 7: METRICS + QUANTITATIVE PLOTS
# =====

def per_sample_overlap_metrics(
    preds: torch.Tensor,
    labels: torch.Tensor,
    num_classes: int,
) -> Tuple[np.ndarray, np.ndarray]:
    pred_oh = labels_to_one_hot(preds, num_classes)
    true_oh = labels_to_one_hot(labels, num_classes)

    reduce_dims = tuple(range(2, pred_oh.ndim))
    inter = (pred_oh * true_oh).sum(dim=reduce_dims).cpu().numpy()
    pred_sum = pred_oh.sum(dim=reduce_dims).cpu().numpy()
    true_sum = true_oh.sum(dim=reduce_dims).cpu().numpy()

    dice_denom = pred_sum + true_sum
    iou_denom = pred_sum + true_sum - inter

    dice = np.where(
        dice_denom > 0,
        (2.0 * inter + 1e-6) / (dice_denom + 1e-6),
        np.nan,
    )
    iou = np.where(
        iou_denom > 0,
        (inter + 1e-6) / (iou_denom + 1e-6),
        np.nan,
```

```

    )
    return dice, iou

class PRCurveAccumulator:
    def __init__(self, thresholds: np.ndarray):
        self.thresholds = thresholds
        self.tp = np.zeros_like(thresholds, dtype=np.int64)
        self.fp = np.zeros_like(thresholds, dtype=np.int64)
        self.fn = np.zeros_like(thresholds, dtype=np.int64)

    def update(self, y_true_binary: np.ndarray, y_score: np.ndarray) -> None:
        y_true_binary = y_true_binary.astype(bool)
        y_score = y_score.astype(np.float32)

        for i, thr in enumerate(self.thresholds):
            y_pred_binary = y_score >= thr
            self.tp[i] += int(np.logical_and(y_pred_binary,
            y_true_binary).sum())
            self.fp[i] += int(np.logical_and(y_pred_binary,
            ~y_true_binary).sum())
            self.fn[i] += int(np.logical_and(~y_pred_binary,
            y_true_binary).sum())

    def as_dataframe(self) -> pd.DataFrame:
        precision = self.tp / np.maximum(self.tp + self.fp, 1)
        recall = self.tp / np.maximum(self.tp + self.fn, 1)
        return pd.DataFrame(
            {
                "threshold": self.thresholds,
                "precision": precision,
                "recall": recall,
                "tp": self.tp,
                "fp": self.fp,
                "fn": self.fn,
            }
        )

@torch.no_grad()
def evaluate_model(
    model: torch.nn.Module,
    val_loader: DataLoader,
    device: torch.device,
    cfg: Config,
) -> Dict[str, object]:
    model.eval()

```

```

cm = np.zeros((cfg.num_classes, cfg.num_classes), dtype=np.int64)
pr_acc = PRCurveAccumulator(thresholds=np.linspace(0.0, 1.0, 101))
hd_metric = HausdorffDistanceMetric(
    include_background=False,
    percentile=cfg.hausdorff_percentile,
    reduction="none",
)

per_sample_rows: List[Dict] = []

for batch in tqdm(val_loader, desc="Final evaluation"):
    images = batch["image"].to(device, non_blocking=True)
    labels = batch["label"].to(device, non_blocking=True).long()
    case_id = batch["case_id"][0]

    logits = infer_logits(model, images, cfg)
    probs = torch.softmax(logits, dim=1)
    preds = torch.argmax(probs, dim=1, keepdim=True)

    dice_np, iou_np = per_sample_overlap_metrics(preds, labels,
cfg.num_classes)

    pred_oh = labels_to_one_hot(preds, cfg.num_classes)
    true_oh = labels_to_one_hot(labels, cfg.num_classes)

    hd_metric(pred_oh, true_oh)
    hd_np = hd_metric.aggregate().cpu().numpy() # [B, C-1]
    hd_metric.reset()

    # Confusion matrix update
    y_true_flat = labels.squeeze(1).cpu().numpy().astype(np.int64).ravel()
    y_pred_flat = preds.squeeze(1).cpu().numpy().astype(np.int64).ravel()
    cm += confusion_matrix(y_true_flat, y_pred_flat,
labels=np.arange(cfg.num_classes))

    # Precision-recall for "electrode vs not-electrode"
    # score = sum of probabilities for classes 1..6
    y_score_elec = probs[:,
ELECTRODE_CLASSES].sum(dim=1).cpu().numpy().ravel()
    y_true_elec = np.isin(y_true_flat, ELECTRODE_CLASSES)
    pr_acc.update(y_true_binary=y_true_elec, y_score=y_score_elec)

    # Per-sample records
    row = {"case_id": case_id}
    for class_id, class_name in enumerate(CLASS_NAMES):
        row[f"dice_{class_name}"] = float(dice_np[0, class_id]) if not
np.isnan(dice_np[0, class_id]) else np.nan
        row[f"iou_{class_name}"] = float(iou_np[0, class_id]) if not

```

```

np.isnan(iou_np[0, class_id]) else np.nan
    if class_id >= 1:
        # hd_np excludes background; class_id 1 maps to hd_np[:,0]
        hd_val = hd_np[0, class_id - 1] if hd_np.ndim == 2 else
hd_np[class_id - 1]
        row[f"hd_{class_name}"] = float(hd_val) if np.isfinite(hd_val)
else np.nan

    row["mean_dice_non_bg"] = float(np.nanmean(dice_np[0, 1:]))
    row["mean_iou_non_bg"] = float(np.nanmean(iou_np[0, 1:]))
    row["mean_dice_electrodes"] = float(np.nanmean(dice_np[0,
ELECTRODE_CLASSES]))
    row["mean_dice_anatomy"] = float(np.nanmean(dice_np[0,
ANATOMY_CLASSES]))
    per_sample_rows.append(row)

per_sample_df = pd.DataFrame(per_sample_rows)

# Per-class summary
per_class_rows = []
for class_id, class_name in enumerate(CLASS_NAMES):
    class_row = {
        "class_id": class_id,
        "class_name": class_name,
        "dice_mean": float(np.nanmean(per_sample_df[f"dice_{class_name}"])),
        "iou_mean": float(np.nanmean(per_sample_df[f"iou_{class_name}"])),
    }
    if class_id >= 1:
        class_row["hausdorff_mean"] =
float(np.nanmean(per_sample_df[f"hd_{class_name}"]))
    else:
        class_row["hausdorff_mean"] = np.nan

    # Precision / recall / F1 from confusion matrix
    tp = cm[class_id, class_id]
    fp = cm[:, class_id].sum() - tp
    fn = cm[class_id, :].sum() - tp
    precision = tp / max(tp + fp, 1)
    recall = tp / max(tp + fn, 1)
    f1 = (2 * precision * recall) / max(precision + recall, 1e-8)
    class_row["precision"] = float(precision)
    class_row["recall"] = float(recall)
    class_row["f1"] = float(f1)

    per_class_rows.append(class_row)

per_class_df = pd.DataFrame(per_class_rows)

```

```

    # Electrode-only metrics
    electrode_df =
per_class_df[per_class_df["class_id"].isin(ELECTRODE_CLASSES)]
    summary_df = pd.DataFrame(
        [
            {"metric": "mean_dice_non_bg", "value":
float(per_sample_df["mean_dice_non_bg"].mean())},
            {"metric": "mean_iou_non_bg", "value":
float(per_sample_df["mean_iou_non_bg"].mean())},
            {"metric": "mean_dice_electrodes", "value":
float(per_sample_df["mean_dice_electrodes"].mean())},
            {"metric": "mean_dice_anatomy", "value":
float(per_sample_df["mean_dice_anatomy"].mean())},
            {"metric": "electrode_macro_precision", "value":
float(electrode_df["precision"].mean())},
            {"metric": "electrode_macro_recall", "value":
float(electrode_df["recall"].mean())},
            {"metric": "electrode_macro_f1", "value":
float(electrode_df["f1"].mean())},
            {"metric": "hausdorff_mean_non_bg", "value":
float(per_class_df[per_class_df["class_id"] > 0]["hausdorff_mean"].mean())},
        ]
    )

    confusion_df = pd.DataFrame(cm, index=CLASS_NAMES, columns=CLASS_NAMES)
    pr_curve_df = pr_acc.as_dataframe()

    return {
        "summary_df": summary_df,
        "per_class_df": per_class_df,
        "per_sample_df": per_sample_df,
        "confusion_df": confusion_df,
        "pr_curve_df": pr_curve_df,
    }

def save_metrics_to_csv(metrics: Dict[str, object], cfg: Config) -> None:
    metrics["summary_df"].to_csv(cfg.metrics_dir / "summary_metrics.csv",
index=False)
    metrics["per_class_df"].to_csv(cfg.metrics_dir / "per_class_metrics.csv",
index=False)
    metrics["per_sample_df"].to_csv(cfg.metrics_dir / "per_sample_metrics.csv",
index=False)
    metrics["confusion_df"].to_csv(cfg.metrics_dir / "confusion_matrix.csv")
    metrics["pr_curve_df"].to_csv(cfg.metrics_dir /
"precision_recall_curve.csv", index=False)

```



```

def plot_training_history(history_df: pd.DataFrame, cfg: Config) -> None:
    # Loss plot
    plt.figure(figsize=(8, 5))
    for stage_name, stage_df in history_df.groupby("stage"):
        plt.plot(stage_df["epoch"], stage_df["train_loss"],
label=f"{stage_name} train loss")
        plt.plot(stage_df["epoch"], stage_df["val_loss"], label=f"{stage_name}
val loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Training and validation loss vs epochs")
    plt.legend()
    plt.tight_layout()
    plt.savefig(cfg.plots_dir / "loss_vs_epochs.png", dpi=200)
    plt.close()

    # Val Dice plot
    plt.figure(figsize=(8, 5))
    for stage_name, stage_df in history_df.groupby("stage"):
        plt.plot(stage_df["epoch"], stage_df["val_dice"], label=f"{stage_name}
val Dice")
    plt.xlabel("Epoch")
    plt.ylabel("Mean validation Dice")
    plt.title("Validation Dice score vs epochs")
    plt.legend()
    plt.tight_layout()
    plt.savefig(cfg.plots_dir / "val_dice_vs_epochs.png", dpi=200)
    plt.close()

def plot_dice_boxplot(per_sample_df: pd.DataFrame, cfg: Config) -> None:
    plt.figure(figsize=(7, 5))
    plt.boxplot(per_sample_df["mean_dice_non_bg"].dropna().values, vert=True)
    plt.ylabel("Mean non-background Dice per validation sample")
    plt.title("Boxplot of Dice scores across validation samples")
    plt.xticks([1], ["Validation set"])
    plt.tight_layout()
    plt.savefig(cfg.plots_dir / "dice_boxplot.png", dpi=200)
    plt.close()

def plot_per_class_dice(per_class_df: pd.DataFrame, cfg: Config) -> None:
    plot_df = per_class_df[per_class_df["class_id"] > 0].copy()
    plt.figure(figsize=(10, 5))
    plt.bar(plot_df["class_name"], plot_df["dice_mean"])
    plt.xlabel("Class")
    plt.ylabel("Mean Dice")
    plt.title("Per-class Dice scores")

```

```

plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(cfg.plots_dir / "per_class_dice.png", dpi=200)
plt.close()

def plot_precision_recall_curve(pr_curve_df: pd.DataFrame, cfg: Config) -> None:
    plt.figure(figsize=(6, 6))
    plt.plot(pr_curve_df["recall"], pr_curve_df["precision"])
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("Precision-recall curve for electrode detection")
    plt.tight_layout()
    plt.savefig(cfg.plots_dir / "precision_recall_curve_electrodes.png",
dpi=200)
    plt.close()

def plot_confusion_matrix_heatmap(confusion_df: pd.DataFrame, cfg: Config) ->
None:
    cm = confusion_df.values.astype(np.float64)

    # Row-normalized view is more readable than raw counts because background
    dominates.
    row_sums = np.maximum(cm.sum(axis=1, keepdims=True), 1.0)
    cm_norm = cm / row_sums

    plt.figure(figsize=(8, 7))
    plt.imshow(cm_norm, interpolation="nearest")
    plt.colorbar()
    plt.xticks(range(len(CLASS_NAMES)), CLASS_NAMES, rotation=45, ha="right")
    plt.yticks(range(len(CLASS_NAMES)), CLASS_NAMES)
    plt.xlabel("Predicted class")
    plt.ylabel("True class")
    plt.title("Pixel-wise confusion matrix heatmap")
    plt.tight_layout()
    plt.savefig(cfg.plots_dir / "confusion_matrix_heatmap.png", dpi=200)
    plt.close()

```

## Visualization of predictions

### Step Eight: Visualization of Predictions

The overlay function below picks the **most informative slice** in each validation case, defined as the slice with the largest union of ground-truth and predicted foreground. That is much more useful for quality control than always taking the central slice, especially for electrode contacts that may appear in only a few axial slices.

```

# =====
# STEP 8: VISUALIZATION OF PREDICTIONS
# =====

@torch.no_grad()
def save_prediction_overlays(
    model: torch.nn.Module,
    val_loader: DataLoader,
    device: torch.device,
    cfg: Config,
    max_cases: Optional[int] = None,
) -> None:
    model.eval()
    max_cases = max_cases or cfg.max_overlay_cases

    saved = 0
    cmap = plt.get_cmap("tab10", cfg.num_classes)

    for batch in tqdm(val_loader, desc="Saving overlays"):
        if saved >= max_cases:
            break

        images = batch["image"].to(device, non_blocking=True)
        labels = batch["label"].to(device, non_blocking=True).long()
        case_id = batch["case_id"][0]

        logits = infer_logits(model, images, cfg)
        preds = torch.argmax(logits, dim=1, keepdim=True)

        image_np = images[0, 0].cpu().numpy()          # [D,H,W]
        label_np = labels[0, 0].cpu().numpy().astype(np.int32)
        pred_np = preds[0, 0].cpu().numpy().astype(np.int32)

        union_fg = ((label_np > 0) | (pred_np > 0)).sum(axis=(1, 2))
        slice_idx = int(np.argmax(union_fg)) if np.max(union_fg) > 0 else
image_np.shape[0] // 2

        img2d = image_np[slice_idx]
        gt2d = label_np[slice_idx]
        pr2d = pred_np[slice_idx]

        fig, axes = plt.subplots(1, 3, figsize=(15, 5))

        axes[0].imshow(img2d, cmap="gray")
        axes[0].set_title(f"{case_id} | Input slice {slice_idx}")
        axes[0].axis("off")

```

```

        axes[1].imshow(img2d, cmap="gray")
        axes[1].imshow(np.ma.masked_where(gt2d == 0, gt2d), cmap=cmap,
alpha=0.40, vmin=0, vmax=cfg.num_classes - 1)
        axes[1].set_title("Ground truth overlay")
        axes[1].axis("off")

        axes[2].imshow(img2d, cmap="gray")
        axes[2].imshow(np.ma.masked_where(pr2d == 0, pr2d), cmap=cmap,
alpha=0.40, vmin=0, vmax=cfg.num_classes - 1)
        axes[2].set_title("Prediction overlay")
        axes[2].axis("off")

    plt.tight_layout()
    out_path = cfg.overlays_dir / f"{case_id}_overlay.png"
    plt.savefig(out_path, dpi=200)
    plt.close(fig)

    saved += 1

```

## Output saving and single-run execution

### Step Nine: Output and Saving

The `main()` function below ties the whole pipeline together. It preprocesses the raw dataset, creates reproducible train and validation splits, trains the supervised model, generates pseudo-labels from unlabeled cases, fine-tunes on the combined pool, evaluates the final model, and saves everything needed for a clean rerun: weights, CSV metrics, plots, and overlays. MONAI is an established PyTorch-based medical imaging framework and is a good fit for exactly this kind of 3D segmentation workflow. <sup>8</sup>

```

# =====
# STEP 9: OUTPUT + SINGLE MAIN EXECUTION BLOCK
# =====

def load_checkpoint_weights(model: torch.nn.Module, checkpoint_path: Path,
device: torch.device) -> None:
    ckpt = torch.load(checkpoint_path, map_location=device)
    model.load_state_dict(ckpt["model_state_dict"])

def main() -> None:
    cfg = Config()
    seed_everything(cfg.seed)
    ensure_dirs(cfg)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

```

```

# -----
# Step 1: preprocess and cache
# -----
cache = build_preprocessed_cache(cfg)
labeled_files = cache["labeled"]
unlabeled_files = cache["unlabeled"]

# -----
# Step 2: train/val split
# -----
train_files, val_files = create_train_val_split(
    labeled_npz_paths=labeled_files,
    seed=cfg.seed,
    val_fraction=0.20,
)

print(f"Labeled train cases: {len(train_files)}")
print(f"Labeled val cases:   {len(val_files)}")
print(f"Unlabeled cases:    {len(unlabeled_files)}")

train_loader = create_train_loader(
    npz_paths=train_files,
    pseudo_flags=[False] * len(train_files),
    cfg=cfg,
)
val_loader = create_val_loader(val_files, cfg)
unlabeled_loader = create_unlabeled_loader(unlabeled_files, cfg)

# -----
# Step 3: model
# -----
model = build_model(cfg).to(device)

# -----
# Step 4: supervised training
# -----
class_weights = compute_class_weights(train_files,
cfg.num_classes).to(device)
loss_fn = build_loss_fn(class_weights=class_weights, cfg=cfg)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=cfg.learning_rate,
    weight_decay=cfg.weight_decay,
)
gpu_aug = build_gpu_train_augment(cfg)

supervised_ckpt = cfg.weights_dir / "best_supervised_model.pth"

```

```

supervised_history = fit_model(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    loss_fn=loss_fn,
    device=device,
    cfg=cfg,
    epochs=cfg.supervised_epochs,
    checkpoint_path=supervised_ckpt,
    stage_name="supervised",
    gpu_aug=gpu_aug,
)

load_checkpoint_weights(model, supervised_ckpt, device)

# -----
# Step 5: pseudo-label generation + fine-tuning
# -----
pseudo_files = generate_pseudo_labels(
    model=model,
    unlabeled_loader=unlabeled_loader,
    device=device,
    cfg=cfg,
)
print(f"Accepted pseudo-labeled cases: {len(pseudo_files)}")

combined_train_files = train_files + pseudo_files
combined_flags = ([False] * len(train_files)) + ([True] * len(pseudo_files))

finetune_loader = create_train_loader(
    npz_paths=combined_train_files,
    pseudo_flags=combined_flags,
    cfg=cfg,
)

# Fine-tuning typically uses a slightly lower LR
finetune_optimizer = torch.optim.Adam(
    model.parameters(),
    lr=cfg.learning_rate * 0.5,
    weight_decay=cfg.weight_decay,
)

finetune_ckpt = cfg.weights_dir / "best_finetuned_model.pth"
finetune_history = fit_model(
    model=model,
    train_loader=finetune_loader,
    val_loader=val_loader,

```

```

        optimizer=finetune_optimizer,
        loss_fn=loss_fn,
        device=device,
        cfg=cfg,
        epochs=cfg.finetune_epochs,
        checkpoint_path=finetune_ckpt,
        stage_name="finetune",
        gpu_aug=gpu_aug,
    )

load_checkpoint_weights(model, finetune_ckpt, device)

# -----
# Steps 6 + 7: final evaluation + plots
# -----
metrics = evaluate_model(
    model=model,
    val_loader=val_loader,
    device=device,
    cfg=cfg,
)
save_metrics_to_csv(metrics, cfg)

full_history = pd.concat([supervised_history, finetune_history],
ignore_index=True)
full_history.to_csv(Path(cfg.work_dir) / "training_history_all_stages.csv",
index=False)

plot_training_history(full_history, cfg)
plot_dice_boxplot(metrics["per_sample_df"], cfg)
plot_per_class_dice(metrics["per_class_df"], cfg)
plot_precision_recall_curve(metrics["pr_curve_df"], cfg)
plot_confusion_matrix_heatmap(metrics["confusion_df"], cfg)

# -----
# Step 8: overlays
# -----
save_prediction_overlays(
    model=model,
    val_loader=val_loader,
    device=device,
    cfg=cfg,
    max_cases=cfg.max_overlay_cases,
)

# -----
# Step 9: final model save
# -----

```

```

final_model_path = cfg.weights_dir / "final_model_weights.pth"
torch.save(
    {
        "model_state_dict": model.state_dict(),
        "class_names": CLASS_NAMES,
        "raw_to_contig": RAW_TO_CONTIG,
        "config": asdict(cfg),
    },
    final_model_path,
)

print(f"Saved final weights to: {final_model_path}")
print(f"Metrics CSVs saved in: {cfg.metrics_dir}")
print(f"Plots saved in: {cfg.plots_dir}")
print(f"Overlays saved in: {cfg.overlays_dir}")

if __name__ == "__main__":
    main()

```

A few implementation choices are worth stating explicitly. First, the pipeline preserves your **electrode-versus-anatomy class ordering** in the training index space even though the raw masks use {4001..4009} IDs. Second, it standardizes **voxel spacing** instead of globally squashing every CT into a single tensor size, because tiny metallic contacts are exactly the kind of structure that suffer when you over-compress the full field of view. Third, the pseudo-labeling stage is intentionally **strict**. With only 86 labeled cases and extremely small positive targets, a conservative pseudo-label pool is usually better than a large noisy one, and that is consistent with the broader uncertainty-aware semi-supervised literature. <sup>9</sup>

If you want the most likely research upgrade after this baseline, I would add a **coarse-to-fine cascade**: first localize the heart or existing lead-containing ROI with a low-resolution network, then run the current multiclass model only inside that cropped region. That recommendation is directly aligned with successful cardiac CT localization-segmentation pipelines in the literature and is especially relevant here because your lead contacts occupy only a tiny fraction of the voxels in each chest CT. <sup>10</sup>

---

<sup>1</sup> <https://www.sciencedirect.com/science/article/abs/pii/S1361841520300220>  
<https://www.sciencedirect.com/science/article/abs/pii/S1361841520300220>

<sup>2</sup> <sup>6</sup> <https://pubmed.ncbi.nlm.nih.gov/33288961/>  
<https://pubmed.ncbi.nlm.nih.gov/33288961/>

<sup>3</sup> <sup>9</sup> <https://research.monash.edu/en/publications/uncertainty-guided-dual-views-for-semi-supervised-volumetric-medi/>  
<https://research.monash.edu/en/publications/uncertainty-guided-dual-views-for-semi-supervised-volumetric-medi/>

<sup>4</sup> <https://www.sciencedirect.com/science/article/pii/S0010482521009859>  
<https://www.sciencedirect.com/science/article/pii/S0010482521009859>



5 <https://www.mdpi.com/2379-139X/11/5/52>

<https://www.mdpi.com/2379-139X/11/5/52>

7 <https://pmc.ncbi.nlm.nih.gov/articles/PMC12286698/>

<https://pmc.ncbi.nlm.nih.gov/articles/PMC12286698/>

8 MONAI - Medical Open Network for AI

[https://project-monai.github.io/?utm\\_source=chatgpt.com](https://project-monai.github.io/?utm_source=chatgpt.com)

10 <https://www.frontiersin.org/journals/cardiovascular-medicine/articles/10.3389/fcvm.2022.983859/full>

<https://www.frontiersin.org/journals/cardiovascular-medicine/articles/10.3389/fcvm.2022.983859/full>